std::nuclear

Содержание

1 Структуры данных 3
1.1 Двумерное дерево Фенвика 3
1.2 Splay-дерево с явным ключом 3
1.3 Splay-дерево с неявным ключом 4
1.4 Link-cut tree 5
1.5 Дерево отрезков с массовыми операциями RECHECK 6
1.6 Декартово дерево (явное/неявное, sum + size) RECHECK 7
1.7 Ordered set/map RECHECK 8
1.8 Дерево Фенвика RECHECK 8
2 Строковые алгоритмы 8
2.1 Динамический Ахо — Корасик 8
2.2 Суффиксный автомат RECHECK 9
2.3 Суффиксное дерево RECHECK 10
2.4 Суффиксный массив и LCP RECHECK 11
2.5 Ахо — Корасик RECHECK 11
2.6 Z-функция RECHECK 12
2.7 Префикс-функция RECHECK 12
3 Геометрия 12
3.1 Точка и векторы RECHECK 12
3.2 Выпуклая оболочка Грэхема O(NlogN) RECHECK 13
4 Графы 13
4.1 Диниц $O(V^2E)$ RECHECK 13
4.2 Min Cost Max Flow (MCMF) RECHECK 14
4.3 Максимальное паросочетание (Кун с таймером) RECHECK 15
4.4 HLD RECHECK 15
4.5 Максимальное паросочетание (Кун, пример с main) RECHECK 17
5 Теория чисел 17
5.1 Расширенный НОД и диофантово уравнение RECHECK 17
5.2 Линейное решето Эратосфена O(n) RECHECK 17
5.3 Проверка на простоту Миллера — Рабина RECHECK 18
5.4 Длинная арифметика (ГПТ) RECHECK 18
6 C++ 19
6.1 Прагмы, генератор случайных чисел и замер времени (ГПТ, надо проверить) RECHECK 19
7 Теоретические справки 20
7.1 Формулы, обоснования и условия применения 20
8 Идеи решения задач 26
8.1 Повторяющиеся приёмы московских контестов 26

1 Структуры данных

1.1 Двумерное дерево Фенвика

```
/// @author arkadiy
template <typename T>
struct FenwickTree2D {
    int n, m;
    vector<vector<T>> t;

    /// @note n ~ x, m ~ y
    FenwickTree2D(int n, int m) : n(n), m(m), t(n + 1, vector<T>(m + 1)) {}

    void Add(int x, int y, int v) {
        x++, y++;
        for (int i = x; i <= n; i |= (i + 1)) {
            for (int j = y; j <= m; j |= (j + 1)) {
                t[i][j] += v;
            }
        }
    }

    T Sum(int x, int y) {
        T s = 0;
        for (int i = x; i > 0; i = (i & (i + 1)) - 1) {
            for (int j = y; j > 0; j = (j & (j + 1)) - 1) {
                s += t[i][j];
            }
        }
        return s;
    }

    T Sum(int x0, int y0, int x1, int y1) {
        x1++, y1++;
        return Sum(x1, y1) - Sum(x0, y1) - Sum(x1, y0) + Sum(x0, y0);
    }
};
```

1.2 Splay-дерево с явным ключом

```
/// @author arkadiy
namespace splay {
struct Node {
    Node *l = nullptr, *r = nullptr, *p = nullptr;
    i64 v = 0;

    Node(i64 v) : v(v) {}
};

void rotate(Node* t) {
    Node* p = t->p;
    Node* g = p->p;

    if (p->l == t) {
        p->l = t->r;
        if (p->l) p->l->p = p;
        t->r = p;
    } else {
        p->r = t->l;
        if (p->r) p->r->p = p;
        t->l = p;
    }

    p->p = t;
    t->p = g;
    if (g) {
        if (g->l == p)
            g->l = t;
        else
            g->r = t;
    }
}

Node* splay(Node* t) {
    while (t && t->p) {
        Node *p = t->p, *g = t->p->p;
        if (!g) {
            rotate(t);
        } else if ((g->l == p) ^ (p->l == t)) {
            rotate(t);
            rotate(t);
        } else {
            rotate(p);
            rotate(t);
        }
    }
    return t;
}

Node* max(Node* t) {
    while (t && t->r) {
        t = t->r;
    }
    splay(t);
    return t;
}

Node* merge(Node* t1, Node* t2) {
    if (!t1) {
        return t2;
    }
    if (!t2) {
        return t1;
    }

    t1 = max(t1);
    t1->r = t2;
    t2->p = t1;

    return t1;
}

Node* find(Node* t, i64 x) {
    if (t == nullptr) {
        return nullptr;
    }

    Node* last = t;
    while (t) {
        last = t;
        if (t->v == x) break;
        if (x < t->v) {
            t = t->l;
        } else {
            t = t->r;
        }
    }

    return splay(last);
}
```

```
pair<Node*, Node*> split(Node* t, i64 x) {
    if (!t) {
        return {nullptr, nullptr};
    }

    t = find(t, x);

    if (t->v >= x) {
        Node* l = t->l;
        t->l = nullptr;
        if (l) {
            l->p = nullptr;
        }

        return {l, t};
    } else {
        Node* r = t->r;
        t->r = nullptr;
        if (r) {
            r->p = nullptr;
        }

        return {t, r};
    }
}
}  // namespace splay
```

1.3 Splay-дерево с неявным ключом

```
/// @author arkadiy
// TODO: refactor :(

struct Node {
    Node* l = nullptr;
    Node* r = nullptr;
    Node* p = nullptr;

    i64 sz = 0, v = 0, mn = 0;

    bool rev = false;  // reverse (lazy prop)
    i64 to_add = 0;    // add (lazy prop)

    Node(i64 v) : v(v), mn(v), sz(1) {}
};

void push(Node* t) {
    if (t->rev) {
        swap(t->l, t->r);
        if (t->l) t->l->rev ^= 1;
        if (t->r) t->r->rev ^= 1;
        t->rev = false;
    }

    t->mn += t->to_add;
    t->v += t->to_add;
    if (t->l) {
        t->l->to_add += t->to_add;
    }
    if (t->r) {
        t->r->to_add += t->to_add;
    }
    t->to_add = 0;
}

void pull(Node* t) {
    push(t);

    t->sz = 1 + (t->l ? t->l->sz : 0) + (t->r ? t->r->sz : 0);
    t->mn = t->v;
    if (t->l) {
        push(t->l);
        t->mn = min(t->mn, t->l->mn);
    }
    if (t->r) {
        push(t->r);
        t->mn = min(t->mn, t->r->mn);
    }
}

void rotate(Node* t) {
    Node* p = t->p;
    Node* g = p->p;

    if (g) push(g);
    push(p);
    push(t);

    if (p->l == t) {
        p->l = t->r;
        if (p->l) p->l->p = p;
        t->r = p;
    } else {
        p->r = t->l;
        if (p->r) p->r->p = p;
        t->l = p;
    }

    p->p = t;
    t->p = g;
    pull(p);
    pull(t);

    if (g) {
        if (g->l == p)
            g->l = t;
        else
            g->r = t;
        pull(g);
    }
}

Node* splay(Node* t) {
    while (t && t->p) {
        Node *p = t->p, *g = t->p->p;
        if (!g) {
            rotate(t);
        } else if ((g->l == p) ^ (p->l == t)) {
            rotate(t);
            rotate(t);
        } else {
            rotate(p);
            rotate(t);
        }
    }
    return t;
}

Node* max(Node* t) {
    if (!t) return nullptr;
    push(t);
```

```
    while (t->r) {
        t = t->r;
        push(t);
    }
    return splay(t);
}

Node* merge(Node* t1, Node* t2) {
    if (!t1) return t2;
    if (!t2) return t1;

    t1 = max(t1);
    t1->r = t2;
    t2->p = t1;
    pull(t1);

    return t1;
}

Node* get(Node* t, i64 i) {
    if (!t || i >= t->sz) {
        return nullptr;
    }

    while (t) {
        push(t);
        i64 leftsz = (t->l ? t->l->sz : 0);
        if (i < leftsz) {
            t = t->l;
        } else if (i == leftsz) {
            return splay(t);
        } else {
            i -= leftsz + 1;
            t = t->r;
        }
    }

    return nullptr;
}

pair<Node*, Node*> split(Node* t, i64 cnt) {
    if (!t) return {nullptr, nullptr};
    if (cnt >= t->sz) return {t, nullptr};

    t = get(t, cnt);

    Node *l = t->l, *r = t;
    if (l) {
        l->p = nullptr;
    }
    if (r) {
        r->l = nullptr;
        pull(r);
    }

    return {l, r};
}

// remove independent subtree (by root)
void remove(Node* t) {
    if (!t) return;
    remove(t->l);
    remove(t->r);
    delete t;
}
```

```
// create new tree with one node
Node* create(i64 v) { return new Node(v); }
```

1.4 Link-cut tree

```
/// @author arkadiy
/// @date 2026-10-17

// TODO: refactor + comments
namespace splay {
const int inf = 1e9;
struct Node {
    Node* l = nullptr;
    Node* r = nullptr;
    Node* p = nullptr;

    i64 sz = 0, v = 0, sm = 0;

    bool rev = false;  // reverse (lazy prop)
    i64 to_add = 0;    // add (lazy prop)

    Node(i64 v) : v(v), sm(v), sz(1) {}
};

bool is_root(Node* t) { return (!t->p || t->p->l != t && t->p->r != t); }

void push(Node* t) {
    if (t->rev) {
        swap(t->l, t->r);
        if (t->l) t->l->rev ^= 1;
        if (t->r) t->r->rev ^= 1;
        t->rev = false;
    }

    t->sm += t->to_add * t->sz;
    t->v += t->to_add;
    if (t->l) {
        t->l->to_add += t->to_add;
    }
    if (t->r) {
        t->r->to_add += t->to_add;
    }
    t->to_add = 0;
}

void pull(Node* t) {
    push(t);

    t->sz = 1 + (t->l ? t->l->sz : 0) + (t->r ? t->r->sz : 0);
    t->sm = t->v;
    if (t->l) {
        push(t->l);
        t->sm += t->l->sm;
    }
    if (t->r) {
        push(t->r);
        t->sm += t->r->sm;
    }
}

void rotate(Node* t) {
    Node* p = t->p;
    Node* g = p->p;
    bool is_p_root = is_root(p);
```

```
    if (!is_p_root) push(g);
    push(p);
    push(t);

    if (p->l == t) {
        p->l = t->r;
        if (p->l) p->l->p = p;
        t->r = p;
    } else {
        p->r = t->l;
        if (p->r) p->r->p = p;
        t->l = p;
    }

    p->p = t;
    t->p = g;
    pull(p);
    pull(t);

    if (!is_p_root) {
        if (g->l == p)
            g->l = t;
        else
            g->r = t;
        pull(g);
    }
}

void push_all(Node* t) {
    if (!is_root(t)) {
        push_all(t->p);
    }
    push(t);
}

Node* splay(Node* t) {
    push_all(t);
    while (!is_root(t)) {
        Node *p = t->p, *g = t->p->p;
        if (is_root(p)) {
            rotate(t);
        } else if ((g->l == p) ^ (p->l == t)) {
            rotate(t);
            rotate(t);
        } else {
            rotate(p);
            rotate(t);
        }
    }
    return t;
}

// create new tree with one node
Node* create(i64 v) { return new Node(v); }

namespace lct {
void expose(Node* v) {
    Node *last = nullptr, *cur = v;
    while (cur != nullptr) {
        splay(cur);
        cur->r = last;
        pull(cur);
        last = cur;
        cur = cur->p;
    }
    splay(v);
}

void make_root(Node* v) {
    expose(v);
    v->rev = true;
    push(v);
}

void link(Node* v, Node* u) {
    make_root(v);
    v->p = u;
}

void cut(Node* v) {
    expose(v);
    Node* u = v->l;
    if (u) {
        u->p = nullptr;
        v->l = nullptr;
        pull(v);
    }
}

void cut(Node* v, Node* u) {
    make_root(v);
    expose(u);
    v->p = nullptr;
    u->l = nullptr;
    pull(u);
}
}  // namespace lct
}  // namespace splay
```

1.5 Дерево отрезков с массовыми операциями RECHECK

```
/// @author Не указан в исходном файле
/// @brief Дерево отрезков с массовыми операциями RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

#include <bits/stdc++.h>
using namespace std;

using ll = long long;

struct SegTree {
    int n = 0;
    vector<ll> t, c;
    static constexpr ll NEG_INF = (ll)-4e18;

    SegTree() = default;
    explicit SegTree(int n_) : n(n_), t(4 * n_, 0), c(4 * n_, 0) {}
    explicit SegTree(const vector<ll>& a) { init(a); }

    void init(int n_) {
        n = n_;
        t.assign(4 * n, 0);
        c.assign(4 * n, 0);
    }

    void init(const vector<ll>& a) {
        n = (int)a.size();
        t.assign(4 * n, 0);
        c.assign(4 * n, 0);
```

```
        build(0, 0, n, a);
    }

    void build(const vector<ll>& a) { init(a); }

    void push(int v) {
        if (c[v] != 0) {
            c[v * 2 + 1] += c[v];
            c[v * 2 + 2] += c[v];
            c[v] = 0;
        }
    }

    void pull(int v) {
        t[v] = max(t[v * 2 + 1] + c[v * 2 + 1],
                   t[v * 2 + 2] + c[v * 2 + 2]);
    }

    void build(int v, int l, int r, const vector<ll>& a) {
        if (r - l == 1) {
            t[v] = a[l];
            c[v] = 0;
            return;
        }
        int m = (l + r) / 2;
        build(v * 2 + 1, l, m, a);
        build(v * 2 + 2, m, r, a);
        t[v] = max(t[v * 2 + 1], t[v * 2 + 2]);
        c[v] = 0;
    }

    void add(int ql, int qr, ll val) { add(0, 0, n, ql, qr, val); }

    void add(int v, int l, int r, int ql, int qr, ll val) {
        if (ql <= l && r <= qr) {
            c[v] += val;
            return;
        }
        if (r <= ql || qr <= l) return;
        int m = (l + r) / 2;
        push(v);
        add(v * 2 + 1, l, m, ql, qr, val);
        add(v * 2 + 2, m, r, ql, qr, val);
        pull(v);
    }

    ll getmax(int ql, int qr) { return getmax(0, 0, n, ql, qr); }

    ll getmax(int v, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr) return t[v] + c[v];
        if (r <= ql || qr <= l) return NEG_INF;
        int m = (l + r) / 2;
        push(v);
        ll left = getmax(v * 2 + 1, l, m, ql, qr);
        ll right = getmax(v * 2 + 2, m, r, ql, qr);
        pull(v);
        return max(left, right);
    }
};
```

1.6 Декартово дерево (явное/неявное, sum + size) RECHECK

```
/// @author Не указан в исходном файле
/// @brief Декартово дерево (явное/неявное, sum + size) RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using ull = unsigned long long;

struct treap {
    struct node {
        ll x, sum, sz; ull y;
        node *l, *r;
        node(): x(0), sum(0), sz(0), y(0), l(nullptr), r(nullptr) {}
    };

    static const int maxn = 300000 + 10;

    node pool[maxn];
    int nodesz = 0;
    mt19937_64 rng;
    node* root = nullptr;

    treap(): rng((uint64_t)chrono::steady_clock::now().time_since_epoch().count()) {}

    node* getnode(ll x = 0) {
        node* p = &pool[nodesz++];
        p->x = x; p->sum = x; p->sz = 1; p->l = p->r = nullptr; p->y = rng();
        return p;
    }

    ll getsum(node* v) const { return v ? v->sum : 0; }
    ll getsz(node* v)  const { return v ? v->sz  : 0; }

    void updatesum(node* v) { v->sum = getsum(v->l) + getsum(v->r) + v->x; }
    void updatesz(node* v)  { v->sz  = getsz(v->l)  + getsz(v->r)  + 1; }

    void update(node* v) {
        if (!v) return;
        updatesz(v);
        updatesum(v);
    }

    // < x , >= x
    void split(node* v, node*& l, node*& r, ll x) {
        if (!v) { l = r = nullptr; return; }
        if (v->x < x) {
            split(v->r, v->r, r, x);
            l = v; update(l);
        } else {
            split(v->l, l, v->l, x);
            r = v; update(r);
        }
    }

    void merge(node*& v, node* l, node* r) {
        if (!l || !r) { v = l ? l : r; return; }
        if (l->y > r->y) {
            merge(l->r, l->r, r);
            v = l; update(v);
        } else {
            merge(r->l, l, r->l);
            v = r; update(v);
        }
    }

    // [1, k] + [k + 1, ... ]
```

```
    void splitk(node* v, node*& l, node*& r, ll k) {
        if (!v || k == 0) { l = nullptr; r = v; return; }
        if (k - getsz(v->l) - 1 >= 0) {
            splitk(v->r, v->r, r, k - getsz(v->l) - 1);
            l = v; update(l);
        } else {
            splitk(v->l, l, v->l, k);
            r = v; update(r);
        }
    }

    void insert(ll x) {
        node *t1 = nullptr, *t3 = nullptr, *t2 = getnode(x);
        split(root, t1, t3, x);
        merge(root, t1, t2);
        merge(root, root, t3);
    }

    void erase_value(ll x) {
        node *t1 = nullptr, *t2 = nullptr, *t3 = nullptr;
        split(root, t1, t2, x);
        split(t2, t2, t3, x + 1);
        merge(root, t1, t3);
    }

    int kth(int k) {
        node *t1 = nullptr, *t2 = nullptr, *t3 = nullptr;
        splitk(root, t2, t3, k);
        splitk(t2, t1, t2, k - 1);
        int res = t2->x;
        merge(root, t1, t2);
        merge(root, root, t3);
        return res;
    }

    void print_inorder(node* v) {
        if (!v) return;
        print_inorder(v->l);
        cout << v->x << ' ';
        print_inorder(v->r);
    }

    int size() const { return (int)getsz(root); }
    ll sum()  const { return getsum(root); }
};
```

1.7 Ordered set/map RECHECK

```
/// @author Не указан в исходном файле
/// @brief Ordered set/map RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

// g++ -std=gnu++17 -O2
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace std;
using namespace __gnu_pbds;

template<class T, class cmp = less<T>>
using ordered_set = tree<T, null_type, cmp, rb_tree_tag, tree_order_statistics_node_update>;

template<class K, class V, class cmp = less<K>>
using ordered_map = tree<K, V, cmp, rb_tree_tag, tree_order_statistics_node_update>;
```

1.8 Дерево Фенвика RECHECK

```
/// @author Не указан в исходном файле
/// @brief Дерево Фенвика RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

struct Fenwick {
   int n;
   vector<ll> f;
   Fenwick() {}
   Fenwick(int n): n(n), f(n+1, 0) {}
   void add(int i, ll v){
       for(; i<=n; i+=i&-i) f[i]+=v;
   }
   ll sumPrefix(int i){
       ll r=0;
       for(; i>0; i-=i&-i) r+=f[i];
       return r;
   }
   ll sumRange(int l, int r){
       if(r<l) return 0;
       return sumPrefix(r)-sumPrefix(l-1);
   }
};
```

2 Строковые алгоритмы

2.1 Динамический Ахо — Корасик

```
/// @author egor
struct AhoCorasick {
    static const int ALPHABET = 26;
    struct Node {
        int next[ALPHABET];    // переходы автомата
        int go[ALPHABET];      // реальные переходы в боре
        int link = 0;          // суфф ссылка
        int up = 0;            // сжатая суфф ссылка (ведёт до ближайшего терминала)
        int cnt = 0;           // кол-во терминальных вершин в поддереве суф ссылок
        int pattern_ind = -1;  // индекс паттерна

        Node() {
            fill(next, next + ALPHABET, -1);
            fill(go, go + ALPHABET, -1);
        }
    };

    vector<Node> nodes;
    bool built = false;

    AhoCorasick() { clear(); }

    inline void clear() {
        nodes.clear();
        nodes.emplace_back();
        built = false;
    }

    inline int get_char(char c) { return c - 'a'; }

    void add_pattern(const string& s, int id = 0) {
        int v = 0;
        for (char c : s) {
            int x = get_char(c);
```

```
            if (nodes[v].go[x] == -1) {
                nodes[v].go[x] = nodes.size();
                nodes.emplace_back();
            }
            v = nodes[v].go[x];
        }
        nodes[v].cnt++;
        nodes[v].pattern_ind = id;
    }

    void build() {
        queue<int> q;
        // обрабатываем соседей корня
        for (int c = 0; c < ALPHABET; ++c) {
            if (nodes[0].go[c] != -1) {
                nodes[0].next[c] = nodes[0].go[c];
                q.push(nodes[0].go[c]);
            } else {
                nodes[0].next[c] = 0;
            }
        }

        while (!q.empty()) {
            int v = q.front();
            q.pop();
            int u = nodes[v].link;

            // проталкиваем терминальность
            nodes[v].up = (nodes[u].cnt > 0) ? u : nodes[u].up;
            nodes[v].cnt += nodes[u].cnt;

            for (int c = 0; c < ALPHABET; ++c) {
                if (nodes[v].go[c] != -1) {
                    // вычисляем суфф ссылку для ребёнка
                    int to = nodes[v].go[c];
                    nodes[v].next[c] = to;
                    nodes[to].link = nodes[u].next[c];
                    q.push(to);
                } else {
                    nodes[v].next[c] = nodes[u].next[c];
                }
            }
        }
        built = true;
    }

    // кол-во совпадений, заканчивающихся в текущем символе
    ll count_matches(int v) const { return nodes[v].cnt; }

    int next_state(int v, char c) { return nodes[v].next[get_char(c)]; }
};

struct DynamicAho {
    struct Layer {
        AhoCorasick aho;
        vector<string> words;

        void clear() {
            aho.clear();
            words.clear();
        }
    };
    vector<Layer> layers;  // инвариант - если layers[i] не пуст, то в нём 2^i строк

    // добавление строки: O(|S| * log N)
    void add_string(const string& s) {
        vector<string> current_words = {s};
        int i = 0;

        // Ищем свободный слой, собираем слова из занятых
        while (i < int(layers.size()) && !layers[i].words.empty()) {
            // сливаем слова на этом уровне
            current_words.insert(current_words.end(), layers[i].words.begin(),
                                 layers[i].words.end());
            layers[i].clear();
            i++;
        }

        // если вышли за пределы вектора, добавляем новый слой
        if (i == int(layers.size())) {
            layers.emplace_back();
        }

        // Строим новый слой (бит становится 1)
        layers[i].words = current_words;
        for (const string& w : layers[i].words) {
            layers[i].aho.add_pattern(w);
        }
        layers[i].aho.build();
    }

    // общее количество вхождений всех паттернов в текст O(|Text| * log N)
    ll count_total_matches(const string& text) {
        ll ans = 0;
        for (auto& layer : layers) {
            if (layer.words.empty()) continue;

            int v = 0;
            for (char c : text) {
                v = layer.aho.next_state(v, c);
                ans += layer.aho.count_matches(v);
            }
        }
        return ans;
    }
};
```

2.2 Суффиксный автомат RECHECK

```
/// @author Не указан в исходном файле
/// @brief Суффиксный автомат RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

struct sufauto {
    vector<map<char, ll>> next;
    vector<ll> len, link;

    vector<ll> cnt;

    sufauto() {
        node();
    }

    int node(){
        next.emplace_back();
        len.push_back(0);
        link.push_back(-1);
        cnt.push_back(0); // cnt[i] = size(endpos i)
        return (int)next.size() - 1;
    }
```

```
    int add(int last, char ch) {
        int new_node = node();
        int sz = len.size();
        cnt[new_node] = 1;
        link[new_node] = 0;
        len[new_node] = len[last] + 1;
        for (int p = last; p != -1; p = link[p]) {
            if (!next[p].count(ch)) {
                next[p][ch] = new_node;
                continue;
            }

            int q = next[p][ch];
            if (len[p] + 1 == len[q]) {
                link[new_node] = q;
                break;
            }

            int clone_q = node();
            link[clone_q] = link[q];
            link[q] = clone_q;
            link[new_node] = clone_q;
            len[clone_q] = len[p] + 1;
            next[clone_q] = next[q];
            for (; p != -1 && next[p][ch] == q; p = link[p]) {
                next[p][ch] = clone_q;
            }
            break;
        }

        return new_node;
    }

    void dfs_cnt(int v, vector<vector<ll>>& gr) {
        for (int u : gr[v]) dfs_cnt(u, gr);
        if (link[v] == -1) return;
        cnt[link[v]] += cnt[v];
    }


    void buildCnt() {
        int sz = len.size();
        vector<vector<ll>> gr(sz);
        for (int i = 1; i < sz; ++i) gr[link[i]].push_back(i); // дерево суф ссылок
        dfs(0, gr, dp);
    }
};
```

2.3 Суффиксное дерево RECHECK

```
/// @author Не указан в исходном файле
/// @brief Суффиксное дерево RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

struct State {
    int v;
    int pos;
    State(int v = 0, int pos = 0) : v(v), pos(pos) {}
};

struct ST {
    vector<map<char, int>> next;
    vector<int> link, par;
    vector<int> l, r;
    State ptr;
    int sz;
    string s;

    int &get(int v, char c) {
        if (!next[v].count(c)) next[v][c] = -1;
        return next[v][c];
    }

    int node() {
        next.emplace_back();
        link.emplace_back(-1);
        par.emplace_back(-1);
        l.emplace_back(0);
        r.emplace_back(0);
        return sz++;
    }

    ST(string s) : ptr(0, 0), sz(0), s(s) {
        node();
        for (int i = 0; i < (int)s.size(); ++i) add(i);
    }

    int len(int v) {
        return r[v] - l[v];
    }

    State go(State st, int from, int to) {
        while (from < to) {
            if (st.pos == len(st.v)) {
                st = State(get(st.v, s[from]), 0);
                if (st.v == -1) return st;
            } else {
                if (s[l[st.v] + st.pos] != s[from]) return {-1, -1};
                if (to - from < len(st.v) - st.pos)
                    return {st.v, st.pos + to - from};
                from += len(st.v) - st.pos;
                st.pos = len(st.v);
            }
        }
        return st;
    }

    int split(State st) {
        if (st.pos == len(st.v)) return st.v;
        if (st.pos == 0) return par[st.v];

        int cur = st.v;
        int mid = node();
        get(par[cur], s[l[cur]]) = mid;
        get(mid, s[l[cur] + st.pos]) = cur;

        l[mid] = l[cur];
        r[mid] = l[cur] + st.pos;
        l[cur] += st.pos;

        par[mid] = par[cur];
        par[cur] = mid;
        return mid;
    }

    int get_link(int v) {
        if (link[v] != -1) return link[v];
        if (par[v] == -1) return 0;
        int to = get_link(par[v]);
```

```
        return link[v] = split(go(State(to, len(to)), l[v] + (par[v] == 0), r[v]));
    }

    void add(int pos) {
        while (true) {
            State nptr = go(ptr, pos, pos + 1);
            if (nptr.v != -1) {
                ptr = nptr;
                return;
            }
            int mid = split(ptr);
            int leaf = node();
            l[leaf] = pos;
            r[leaf] = (int)s.size();
            par[leaf] = mid;
            get(mid, s[pos]) = leaf;

            ptr.v = get_link(mid);
            ptr.pos = len(ptr.v);
            if (!mid) break;
        }
    }
};
```

2.4 Суффиксный массив и LCP RECHECK

```
/// @author Не указан в исходном файле
/// @brief Суффиксный массив и LCP RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

struct SuffixArray {
    int n;
    string s;
    vector<ll> sa;
    SuffixArray(const string& _s) {
        n = _s.size();
        s = _s;
        sa = suffix_array(s);
    }
    vector<ll> sort_cyclic_shifts(const string& s) {
        int n = s.size();
        const int alphabet = 256;
        vector<ll> p(n), c(n), cnt(max(alphabet, n));
        for (unsigned char el: s) cnt[el]++;
        for (int i = 1; i < alphabet; ++i) cnt[i] += cnt[i - 1];
        for (int i = 0; i < n; ++i) p[--cnt[(unsigned char)s[i]]] = i;
        c[p[0]] = 0;
        int classes = 1;
        for (int i = 1; i < n; ++i) {
            if (s[p[i]] != s[p[i - 1]]) ++classes;
            c[p[i]] = classes - 1;
        }
        vector<ll> pn(n), cn(n);
        for (int h = 0; (1 << h) < n; ++h) {
            for (int i = 0; i < n; i++) {
                pn[i] = p[i] - (1 << h);
                if (pn[i] < 0) pn[i] += n;
            }
            fill(cnt.begin(), cnt.begin() + classes, 0);
            for (auto el: pn) cnt[c[el]]++;
            for (int i = 1; i < classes; ++i) cnt[i] += cnt[i - 1];
            for (int i = n - 1; i >= 0; --i) p[--cnt[c[pn[i]]]] = pn[i];
            cn[p[0]] = 0;
            classes = 1;
            for (int i = 1; i < n; i++) {
                pair<int,int> cur = {c[p[i]], c[(p[i] + (1 << h)) % n]};
                pair<int,int> prev = {c[p[i - 1]], c[(p[i - 1] + (1 << h)) % n]};
                if (cur != prev) ++classes;
                cn[p[i]] = classes - 1;
            }
            c.swap(cn);
        }
        return p;
    }
    vector<ll> suffix_array(string s) {
        s += char(0);
        vector<ll> sorted_shifts = sort_cyclic_shifts(s);
        sorted_shifts.erase(sorted_shifts.begin());
        return sorted_shifts;
    }
};

vector<ll> build_lcp(const string& s, const vector<ll>& sa) {
    int n = s.size();
    vector<ll> rank(n), lcp(n - 1);
    for (int i = 0; i < n; ++i) rank[sa[i]] = i;
    int k = 0;
    for (int i = 0; i < n; ++i) {
        int r = rank[i];
        if (r == n - 1) {
            k = 0;
            continue;
        }
        int j = sa[r + 1];
        while (i + k < n && j + k < n && s[i + k] == s[j + k]) ++k;
        lcp[r] = k;
        if (k) --k;
    }
    return lcp;
}
```

2.5 Ахо — Корасик RECHECK

```
/// @author Не указан в исходном файле
/// @brief Ахо — Корасик RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

struct Aho {
    const int A = 26;
    int node_cnt, pattern_cnt;
    vector<vector<int>> next;
    vector<vector<int>> terminals;
    vector<int> suf_link, terminal_link;

    Aho() : node_cnt(0), pattern_cnt(0) { node(); }

    int node() {
        next.emplace_back(A);
        terminals.emplace_back();
        suf_link.emplace_back();
        terminal_link.emplace_back();
        return node_cnt++;
    }

    inline int get(char c) {
        return c - 'a';
    }
```

```
    int add_pattern(const string &s) {
        int v = 0;
        for (auto c: s) {
            if (!next[v][get(c)]) next[v][get(c)] = node();
            v = next[v][get(c)];
        }
        terminals[v].emplace_back(pattern_cnt);
        return pattern_cnt++;
    }

    void build() {
        queue<int> q;
        q.push(0);
        while (!q.empty()) {
            int par = q.front();
            q.pop();
            for (int c = 0; c < A; ++c) {
                int child = next[par][c];
                if (!child) {
                    next[par][c] = next[suf_link[par]][c];
                    continue;
                }
                // set suffix link
                if (par == 0) suf_link[child] = 0;
                else suf_link[child] = next[suf_link[par]][c];

                // set terminal link
                if (terminals[suf_link[child]].empty()) terminal_link[child] =
terminal_link[suf_link[child]];
                else terminal_link[child] = suf_link[child];

                q.push(child);
            }
        }
    }

    int go(int v, char c) {
        return next[v][get(c)];
    }

    // get all matches ending at given position
    vector<int> get_matches(int v) {
        vector<int> res;
        while (v) {
            for (auto match: terminals[v]) res.emplace_back(match);
            v = terminal_link[v];
        }
        return res;
    }
};
```

2.6 Z-функция RECHECK

```
/// @author Не указан в исходном файле
/// @brief Z-функция RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

vector<int> zfunc(const string& s) {
    int n = s.size();
    vector<int> z(n);
    int j = 0;
    for (int i = 1; i < n; ++i) {
        z[i] = max(min(z[i - j], z[j] + j - i), 0);
        while (i + z[i] < n && s[i + z[i]] == s[z[i]]) {
            int c1 = z[i], c2 = i + z[i];
            z[i]++;
        }
        if (i + z[i] > j + z[j]) {
            j = i;
        }
    }
    z[0] = n;
    return v;
}
```

2.7 Префикс-функция RECHECK

```
/// @author Не указан в исходном файле
/// @brief Префикс-функция RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

vector<int> preffunc(const string& s) {
    int n = s.size();
    vector<int> p(n);

    for (int i = 1; i < n; ++i) {
        int j = p[i - 1];
        while (j != 0 && s[j] != s[i]) {
            j = p[j - 1];
        }
        if (s[j] == s[i]) {
            j++;
        }
        p[i] = j;
    }
    return p;
}
```

3 Геометрия

3.1 Точка и векторы RECHECK

```
/// @author Не указан в исходном файле
/// @brief Точка и векторы RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

const ld pi = acos(-1);
const ld eps = 1e-12;

struct point {
    ld x, y;
    point() {}
    point(ld x, ld y): x(x), y(y) {}
    point operator +(const point& a) const {
        return point(x + a.x, y + a.y);
    }
    point operator -(const point& a) const {
        return point(x - a.x, y - a.y);
    }
    ld sq2() {
        return x * x + y * y;
    }
    ld len() {
        return sqrt(sq2());
    }
```

```
    ld operator *(const point& a) {
        return x * a.x + y * a.y;
    }
    point operator *(ll val) {
        return point(x * val, y * val);
    }
    ld operator %(const point& a) { // векторное произведение
        return x * a.y - y * a.x;
    }
    bool operator ==(const point& a) const {
        return eq(x, a.x) && eq(y, a.y);
    }
    point rotate90() {
        return point(y, -x);
    }
    point rotate270() {
        return point(-y, x);
    }
    point rot(ld angle) {
        return point(x * cos(angle) - y * sin(angle), x * sin(angle) + y * cos(angle));
    }
    point norm() {
        return point(x / len(), y / len());
    }
    point resize(ld val) {
        return point(x * val, y * val);
    }
};
```

3.2 Выпуклая оболочка Грэхема O(NlogN) RECHECK

```
/// @author Не указан в исходном файле
/// @brief Выпуклая оболочка Грэхема O(NlogN) RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

struct r
{
    ld x, y;
    r(ld a, ld b) : x(a), y(b) {}
    r() : x(0), y(0) {}
    ld len() {return hypot(x, y);}
};

ld operator^(r a, r b) {return a.x * b.y - b.x * a.y;}
r operator-(r a, r b) {return {a.x - b.x, a.y - b.y};}
bool operator==(r a, r b) {return a.x == b.x && a.y == b.y;}

vector<r> graham(vector<r> points) {
    r p0(points[0].x, points[0].y);
    for (r p : points) {
        if (p.y < p0.y || p.y == p0.y && p.x < p0.x) {
            p0 = p;
        }
    }

    sort(points.begin(), points.end(), [&] (r a, r b) {
        return abs((a - p0) ^ (b - p0)) < EPS && (a - p0).len() > (b - p0).len() || ((a - p0) ^ (b -
p0)) > 0;
    });

    vector<r> hull;
    for (r p : points) {
        while (hull.size() >= 2) {
            r new_vec = p - hull.back();
            r last_vec = hull.back() - hull[hull.size() - 2];
            if ((last_vec ^ new_vec) < 0 || abs(last_vec ^ new_vec) < EPS)
                hull.pop_back();
            else
                break;
        }
        hull.push_back(p);
    }

    return hull;
}
```

4 Графы

4.1 Диниц O(V²E) RECHECK

```
/// @author Не указан в исходном файле
/// @brief Диниц O(V²E) RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

#include <bits/stdc++.h>
using namespace std;

struct Dinic {
    using ll = long long;

    struct Edge {
        int to;
        ll cap;
        int rev;
    };

    int n;
    vector<vector<Edge>> g;
    vector<int> level;
    vector<int> it;

    Dinic(int n) : n(n), g(n), level(n), it(n) {}

    void add_edge(int u, int v, ll cap) {
        Edge a{v, cap, (int)g[v].size()};
        Edge b{u, 0,   (int)g[u].size()};
        g[u].push_back(a);
        g[v].push_back(b);
    }

    bool bfs(int s, int t) {
        fill(level.begin(), level.end(), -1);
        queue<int> q;
        level[s] = 0;
        q.push(s);
        while (!q.empty()) {
            int v = q.front(); q.pop();
            for (const Edge &e : g[v]) {
                if (e.cap > 0 && level[e.to] < 0) {
                    level[e.to] = level[v] + 1;
                    q.push(e.to);
                }
            }
        }
        return level[t] >= 0;
    }
```

```
    ll dfs(int v, int t, ll f) {
        if (v == t) return f;
        for (int &i = it[v]; i < (int)g[v].size(); ++i) {
            Edge &e = g[v][i];
            if (e.cap > 0 && level[e.to] == level[v] + 1) {
                ll pushed = dfs(e.to, t, min(f, e.cap));
                if (pushed > 0) {
                    e.cap -= pushed;
                    g[e.to][e.rev].cap += pushed;
                    return pushed;
                }
            }
        }
        return 0;
    }

    ll max_flow(int s, int t) {
        const ll INF = (ll)4e18;
        ll flow = 0;
        while (bfs(s, t)) {
            fill(it.begin(), it.end(), 0);
            ll f;
            while ((f = dfs(s, t, INF)) > 0) {
                flow += f;
            }
        }
        return flow;
    }
};
```

4.2 Min Cost Max Flow (MCMF) RECHECK

```
/// @author Не указан в исходном файле
/// @brief Min Cost Max Flow (MCMF) RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

struct MCMF {
    using ll = long long;
    static const ll INF = (1LL << 61);

    struct Edge {
        int from, to;
        ll cap, cost;
        Edge(int _from, int _to, ll _cap, ll _cost)
            : from(_from), to(_to), cap(_cap), cost(_cost) {}
    };

    int n, s, t;
    ll flow, min_cost;
    vector<vector<int>> g;
    vector<Edge> e;
    vector<ll> d, potential;
    vector<int> par;
    bool neg;

    MCMF(int _n = 0) { // 0-based
        init(_n);
    }

    void init(int _n) {
        n = _n + 10;             // небольшой запас, как у тебя
        g.assign(n, {});
        e.clear();
        potential.assign(n, 0);
        neg = false;
    }

    void add_edge(int u, int v, ll cap, ll cost, bool directed = true) {
        if (cost < 0) neg = true;
        int id = (int)e.size();
        g[u].push_back(id);
        e.emplace_back(u, v, cap, cost);
        g[v].push_back(id ^ 1);
        e.emplace_back(v, u, 0, -cost);
        if (!directed)
            add_edge(v, u, cap, cost, true);
    }

    bool dijkstra() {
        par.assign(n, -1);
        d.assign(n, INF);

        using P = pair<ll, int>;
        priority_queue<P, vector<P>, greater<P>> q;

        d[s] = 0;
        q.emplace(0, s);

        while (!q.empty()) {
            auto [dist, v] = q.top();
            q.pop();
            if (dist != d[v]) continue;
            for (int id : g[v]) {
                const Edge &ed = e[id];
                if (ed.cap <= 0) continue;
                int to = ed.to;
                ll w = ed.cost + potential[v] - potential[to];
                if (d[v] + w < d[to]) {
                    d[to] = d[v] + w;
                    par[to] = id;
                    q.emplace(d[to], to);
                }
            }
        }

        if (d[t] == INF) return false;

        // стандартное обновление потенциалов
        for (int i = 0; i < n; i++) {
            if (d[i] < INF)
                potential[i] += d[i];
        }

        return true;
    }

    ll send_flow(int v, ll cur) {
        if (par[v] == -1) return cur;
        int id = par[v];
        int from = e[id].from;
        ll f = send_flow(from, min(cur, e[id].cap));
        e[id].cap     -= f;
        e[id ^ 1].cap += f;
        min_cost      += f * e[id].cost; // оригинальная стоимость
        return f;
    }

    // возвращает {maxflow, mincost}
    pair<ll, ll> solve(int _s, int _t, ll goal = INF) {
        s = _s;
```

```
        t = _t;
        flow = 0;
        min_cost = 0;
        potential.assign(n, 0);

        if (neg) {
            // Bellman–Ford для стартовых потенциалов
            d.assign(n, INF);
            d[s] = 0;
            bool relax = true;
            for (int it = 0; it < n && relax; ++it) {
                relax = false;
                for (int v = 0; v < n; ++v) {
                    if (d[v] == INF) continue;
                    for (int id : g[v]) {
                        const Edge &ed = e[id];
                        if (ed.cap <= 0) continue;
                        int to = ed.to;
                        if (d[to] > d[v] + ed.cost) {
                            d[to] = d[v] + ed.cost;
                            relax = true;
                        }
                    }
                }
            }
            for (int i = 0; i < n; i++)
                if (d[i] < INF)
                    potential[i] = d[i];
        }

        while (flow < goal && dijkstra()) {
            flow += send_flow(t, goal - flow);
        }
        return {flow, min_cost};
    }
};
```

4.3 Максимальное паросочетание (Кун с таймером) RECHECK

```
/// @author Не указан в исходном файле
/// @brief Максимальное паросочетание (Кун с таймером) RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

vector<int> g[N]; int mt[N];
int last_version[N]; int timer = 0;
bool try_Kuhn(int v) {
    if (last_version[v] == timer) return false;
    last_version[v] = timer;
    for (int u : g[v]) {
        if (mt[u] == -1) {
            mt[u] = v;
            return true;
        }
        if (try_Kuhn(mt[u])) {
            mt[u] = v;
            return true;
        }
    }
    return false;
}

void solve() {
    int cnt = 0;
    fill(mt, mt + N, -1);
    for (int v = 1; v <= n; v++) {
        timer++;
        if (try_Kuhn(v)) {
            cnt++;
        }
    }
    cout << cnt << "\n";
    for (int u = 1; u <= m; u++) {
        if (mt[u] != -1) {
            // вывод -- паросочетание
            cout << mt[u] << " " << u << "\n";
        }
    }
}
```

4.4 HLD RECHECK

```
/// @author Не указан в исходном файле
/// @brief HLD RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

int n, q;
vector<int> g[N];
ll vertex_value[N];
int sz[N];
int par[N];

void calc_sz(int v, int p_node = 0) {
    par[v] = p_node;
    sz[v] = 1;

    int heavy_child_candidate = -1;
    int max_sz_child = 0;

    for (int i = 0; i < g[v].size(); ++i) {
        int& u = g[v][i];
        if (u == p_node) continue;

        calc_sz(u, v);
        sz[v] += sz[u];

        if (sz[u] > max_sz_child) {
            max_sz_child = sz[u];
        }
    }

    for (int i = 0; i < g[v].size(); ++i) {
        int u = g[v][i];
        if (u == p_node) continue;
        if (sz[u] == max_sz_child && max_sz_child > 0) {
            swap(g[v][0], g[v][i]);
            break;
        }
    }
    if (!g[v].empty() && g[v][0] == p_node) {
        for(int i = 1; i < g[v].size(); ++i) {
            if(g[v][i] != p_node) {
                swap(g[v][0], g[v][i]);
                break;
            }
        }
    }
}
```

```
int timer;
int tin[N];
int tout[N];
int head[N];

void mark_vertexes(int v, int p_node, bool heavy_way) {
    tin[v] = timer++;
    if (heavy_way) {
        head[v] = head[p_node];
    } else {
        head[v] = v;
    }

    if (!g[v].empty() && g[v][0] != p_node) {
        mark_vertexes(g[v][0], v, true);
    }

    for (int i = 0; i < g[v].size(); ++i) {
        int u = g[v][i];
        if (u == p_node || u == g[v][0]) continue;
        mark_vertexes(u, v, false);
    }
    tout[v] = timer;
}

class Node {
public:
    ll value;
    Node(ll x = 0) : value(x) {}
};

ostream& operator<<(ostream& out, const Node& nd) {
    out << nd.value;
    return out;
}

Node nll_node(0);

Node t[2 * N];
Node a[N + 1];

Node combine(Node l, Node r) {
    return Node(l.value + r.value);
}

void build(int v_seg, int vl, int vr) {
    if (vl + 1 == vr) {
        t[v_seg] = a[vl];
        return;
    }
    int vm = vl + (vr - vl) / 2;
    build(2 * v_seg, vl, vm);
    build(2 * v_seg + 1, vm, vr);
    t[v_seg] = combine(t[2 * v_seg], t[2 * v_seg + 1]);
}

void update(int v_seg, int vl, int vr, int index_to_update, Node new_val_node) {
    if (vl + 1 == vr) {
        t[v_seg] = new_val_node;
        return;
    }
    int vm = vl + (vr - vl) / 2;
    if (index_to_update < vm) {
        update(2 * v_seg, vl, vm, index_to_update, new_val_node);
    } else {
        update(2 * v_seg + 1, vm, vr, index_to_update, new_val_node);
    }
    t[v_seg] = combine(t[2 * v_seg], t[2 * v_seg + 1]);
}

Node get(int v_seg, int vl, int vr, int ql, int qr_inclusive) {
    if (ql > qr_inclusive) return nll_node;

    if (ql <= vl && (vr - 1) <= qr_inclusive) {
        return t[v_seg];
    }
    if ((vr - 1) < ql || qr_inclusive < vl) {
        return nll_node;
    }

    int vm = vl + (vr - vl) / 2;
    Node left_res = get(2 * v_seg, vl, vm, ql, qr_inclusive);
    Node right_res = get(2 * v_seg + 1, vm, vr, ql, qr_inclusive);
    return combine(left_res, right_res);
}

bool ancestor(int v_anc, int u_desc) {
    if (v_anc == 0) return true;
    if (u_desc == 0) return false;
    return tin[v_anc] <= tin[u_desc] && tin[u_desc] < tout[v_anc];
}

ll get_way_value(int cur_a, int cur_b) {
    ll result = 0;

    int original_a = cur_a;
    int original_b = cur_b;

    while (!ancestor(head[cur_a], original_b)) {
        if (cur_a == 0) break;
        result += get(1, 1, n + 1, tin[head[cur_a]], tin[cur_a]).value;
        cur_a = par[head[cur_a]];
    }

    while (!ancestor(head[cur_b], original_a)) {
        if (cur_b == 0) break;
        result += get(1, 1, n + 1, tin[head[cur_b]], tin[cur_b]).value;
        cur_b = par[head[cur_b]];
    }

    if (tin[cur_a] > tin[cur_b]) {
        swap(cur_a, cur_b);
    }
    result += get(1, 1, n + 1, tin[cur_a], tin[cur_b]).value;

    return result;
}

void build_hld() {
    calc_sz(1, 0);

    timer = 1;
    mark_vertexes(1, 0, false);

    for (int i = 1; i <= n; ++i) {
        a[tin[i]] = Node(vertex_value[i]);
    }
    build(1, 1, n + 1);
}

void solve() {
    cin >> n >> q;
```

```
    for (int i = 0; i < n; ++i) {
        cin >> vertex_value[i + 1];
    }

    for (int i = 0; i < n - 1; ++i) {
        int v, u;
        cin >> v >> u;
        v++; u++;
        g[v].push_back(u);
        g[u].push_back(v);
    }

    build_hld();

    while (q--) {
        int type;
        cin >> type;
        if (type == 0) {
            int ind, x;
            cin >> ind >> x;
            ind++;

            vertex_value[ind] += (ll)x;
            update(1, 1, n + 1, tin[ind], Node(vertex_value[ind]));
        } else {
            int v, u;
            cin >> v >> u;
            v++; u++;
            cout << get_way_value(v, u) << "\n";
        }
    }
}
```

4.5 Максимальное паросочетание (Кун, пример с main) RECHECK

```
/// @author Не указан в исходном файле
/// @brief Максимальное паросочетание (Кун, пример с main) RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

int n, k;
vector < vector<int> > g;
vector<int> mt;
vector<char> used;

bool try_kuhn (int v) {
    if (used[v])  return false;
    used[v] = true;
    for (size_t i=0; i<g[v].size(); ++i) {
        int to = g[v][i];
        if (mt[to] == -1 || try_kuhn (mt[to])) {
            mt[to] = v;
            return true;
        }
    }
    return false;
}

int main() {
    ... чтение графа ...

    mt.assign (k, -1);
    for (int v=0; v<n; ++v) {
        used.assign (n, false);
        try_kuhn (v);
    }

    for (int i=0; i<k; ++i)
        if (mt[i] != -1)
            printf ("%d %d\n", mt[i]+1, i+1);
}
```

5 Теория чисел

5.1 Расширенный НОД и диофантово уравнение RECHECK

```
/// @author Не указан в исходном файле
/// @brief Расширенный НОД и диофантово уравнение RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

ll gcd(ll a, ll b, ll& x, ll& y) { // ax + by = c
    if (a == 0) {
        x = 0, y = 1;
        return b;
    }

    ll x1, y1;
    ll g = gcd(b % a, a, x1, y1);
    x = y1 - (b / a) * x1;
    y = x1;
    return g;
}
```

5.2 Линейное решето Эратосфена O(n) RECHECK

```
/// @author Не указан в исходном файле
/// @brief Линейное решето Эратосфена O(n) RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

struct linear_sieve {
    int n = 0;
    vector<int> lp, primes;

    void init(int n_) {
        n = n_;
        lp.assign(n + 1, 0);
        primes.clear();
        for (llt d = 2; d <= n; ++d) {
            if (lp[d] == 0) {
                lp[d] = d;
                primes.push_back(d);
            }
            for (ll i = 0; i < (int)primes.size() && primes[i] <= lp[d] && 1LL * primes[i] * d <= n; +
+i) {
                lp[d * primes[i]] = primes[i];
            }
        }
    }

    bool is_prime(int x) const {
        return x >= 2 && x <= n && lp[x] == x;
    }

    vector<pair<int,int>> factorize(int x) const {
```

```
        vector<pair<int,int>> res;
        while (x > 1) {
            int p = lp[x], c = 0;
            while (x % p == 0) { x /= p; ++c; }
            res.push_back({p, c});
        }
        return res;
    }
};
```

5.3 Проверка на простоту Миллера — Рабина RECHECK

```
/// @author Не указан в исходном файле
/// @brief Проверка на простоту Миллера — Рабина RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

// Быстрое возведение в степень по модулю с защитой от переполнения
long long power(long long base, long long exp, long long mod) {
    long long res = 1;
    base %= mod;
    while (exp > 0) {
        if (exp % 2 == 1) res = (__int128)res * base % mod;
        base = (__int128)base * base % mod;
        exp /= 2;
    }
    return res;
}

// Тест Миллера-Рабина
bool is_prime(long long n) {
    if (n < 2) return false;
    if (n == 2 || n == 3) return true;
    if (n % 2 == 0 || n % 3 == 0) return false;

    long long d = n - 1;
    while (d % 2 == 0) {
        d /= 2;
    }

    // Свидетели, достаточные для детерминированной проверки всех long long
    for (long long a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
        if (n == a) return true;

        long long x = power(a, d, n);
        if (x == 1 || x == n - 1) continue;

        bool composite = true;
        long long temp_d = d;
        while (temp_d * 2 < n - 1) {
            temp_d *= 2;
            x = (__int128)x * x % n;
            if (x == n - 1) {
                composite = false;
                break;
            }
        }
        if (composite) return false;
    }
    return true;
}
```

5.4 Длинная арифметика (ГПТ) RECHECK

```
/// @author Не указан в исходном файле
/// @brief Длинная арифметика (ГПТ) RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

#include <bits/stdc++.h>
using namespace std;

class Big {
    static constexpr int BASE = 1'000'000'000;
    vector<int> d;  // Младший блок первым. Пустой вектор означает 0.

    void trim() {
        while (!d.empty() && d.back() == 0) d.pop_back();
    }

public:
    explicit Big(unsigned long long x = 0) {
        for (; x; x /= BASE) d.push_back(x % BASE);
    }

    // s состоит из десятичных цифр. Ведущие нули допустимы.
    explicit Big(const string& s) {
        for (int r = int(s.size()); r > 0; r -= 9) {
            int l = max(0, r - 9);
            d.push_back(stoi(s.substr(l, r - l)));
        }
        trim();
    }

    friend istream& operator>>(istream& in, Big& a) {
        string s;
        if (in >> s) a = Big(s);
        return in;
    }

    friend ostream& operator<<(ostream& out, const Big& a) {
        if (a.d.empty()) return out << '0';
        out << a.d.back();
        char old_fill = out.fill('0');
        for (int i = int(a.d.size()) - 2; i >= 0; --i)
            out << setw(9) << a.d[i];
        out.fill(old_fill);
        return out;
    }

    // -1, 0 или 1 для a < b, a == b или a > b.
    friend int cmp(const Big& a, const Big& b) {
        if (a.d.size() != b.d.size())
            return a.d.size() < b.d.size() ? -1 : 1;
        for (int i = int(a.d.size()) - 1; i >= 0; --i) {
            if (a.d[i] != b.d[i]) return a.d[i] < b.d[i] ? -1 : 1;
        }
        return 0;
    }

    friend Big operator+(const Big& a, const Big& b) {
        Big c;
        int n = int(max(a.d.size(), b.d.size()));
        int carry = 0;
        for (int i = 0; i < n || carry; ++i) {
            int cur = carry;
            if (i < int(a.d.size())) cur += a.d[i];
            if (i < int(b.d.size())) cur += b.d[i];
```

```
        c.d.push_back(cur % BASE);
        carry = cur / BASE;
    }
    return c;
}

// Требуется a >= b.
friend Big operator-(Big a, const Big& b) {
    assert(cmp(a, b) >= 0);
    int borrow = 0;
    for (int i = 0; i < int(a.d.size()); ++i) {
        int cur = a.d[i] - borrow;
        if (i < int(b.d.size())) cur -= b.d[i];
        borrow = cur < 0;
        a.d[i] = cur + borrow * BASE;
    }
    a.trim();
    return a;
}

friend Big operator*(const Big& a, const Big& b) {
    if (a.d.empty() || b.d.empty()) return Big();
    int n = int(a.d.size()), m = int(b.d.size());
    Big c;
    c.d.assign(n + m, 0);
    for (int i = 0; i < n; ++i) {
        long long carry = 0;
        for (int j = 0; j < m; ++j) {
            long long cur = c.d[i + j] + 1LL * a.d[i] * b.d[j] + carry;
            c.d[i + j] = int(cur % BASE);
            carry = cur / BASE;
        }
        c.d[i + m] = int(carry);
    }
    c.trim();
    return c;
}

// b > 0. Возвращает {частное, остаток}.
pair<Big, uint32_t> divmod(uint32_t b) const {
    assert(b > 0);
    Big q;
    q.d.resize(d.size());
    uint64_t rem = 0;
    for (int i = int(d.size()) - 1; i >= 0; --i) {
        uint64_t cur = rem * BASE + d[i];
        q.d[i] = int(cur / b);
        rem = cur % b;
    }
    q.trim();
    return {move(q), uint32_t(rem)};
}

// b > 0. Деление столбиком с бинпоиском очередного блока частного.
pair<Big, Big> divmod(const Big& b) const {
    assert(!b.d.empty());
    if (b.d.size() == 1) {
        auto [q, r] = divmod(uint32_t(b.d[0]));
        return {move(q), Big(r)};
    }

    Big q, r;
    q.d.resize(d.size());
    for (int i = int(d.size()) - 1; i >= 0; --i) {
        // r = r * BASE + d[i].
        if (!r.d.empty() || d[i] != 0) r.d.insert(r.d.begin(), d[i]);
        if (cmp(r, b) < 0) continue;

        int lo = 1, hi = BASE;
        while (hi - lo > 1) {
            int mid = (lo + hi) / 2;
            if (cmp(b * Big(mid), r) <= 0) lo = mid;
            else hi = mid;
        }
        q.d[i] = lo;
        r = r - b * Big(lo);
    }
    q.trim();
    return {move(q), move(r)};
  }
};
```

6 C++

6.1 Прагмы, генератор случайных чисел и замер времени (ГПТ, надо проверить) RECHECK

```
/// @author Не указан в исходном файле
/// @brief Прагмы, генератор случайных чисел и замер времени (ГПТ, надо проверить) RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

// #pragma GCC optimize("O3")           // Доп. оптимизации и векторизация.
// #pragma GCC optimize("unroll-loops") // Разворачивание подходящих циклов.
// #pragma GCC target("avx2")           // Только если на сервере есть AVX2.

// Прагма действует только на следующий цикл.
#pragma GCC unroll 4
for (int i = 0; i < n; ++i) {
    sum += a[i];
}

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
// а уже считан. Включить таймер при сборке флагом -DLOCAL.
#ifdef LOCAL
const auto start = chrono::steady_clock::now();
#endif

ll sum = 0;
for (int x : a) {
    sum += x;  // Измеряемый код.
}

#ifdef LOCAL
const auto elapsed = chrono::steady_clock::now() - start;
const double ms = chrono::duration<double, milli>(elapsed).count();
cerr << "loop: " << ms << " ms\n";  // Вывод уже вне замера.
#endif
// g++ -DLOCAL main.cpp
```

7 Теоретические справки

7.1 Формулы, обоснования и условия применения

Как пользоваться справочником

В карточках собраны формулы, обоснования и условия применения. Коды T01 и далее связывают их с полной картой задач; специальные методы лежат в дополнении и не включены в печатную часть.

T01. КТО, расширенный НОД и остатки суммы цифр

Распознавание. Неизвестное целое восстанавливается по нескольким остаткам; периодические события должны совпасть; запрос возвращает сумму цифр в выбранном основании. Примеры: 2023 B, арифметическая часть 2018 E.

Расширенный Евклид находит `g=gcd(a,b)` и `u,v`, для которых `au+bv=g`, за `O(log min(|a|,|b|))`. Уравнение `ax+by=c` разрешимо тогда и только тогда, когда `g|c`. Если `(x0,y0)` — решение, то все решения:

$$x = x_0 + t\frac{b}{g}, \qquad y = y_0 - t\frac{a}{g}, \quad t \in \mathbb{Z}.$$

Обобщённая КТО. Объединяем `x≡a (mod m)` и `x≡b (mod n)`, где `m,n>0`. Подставим `x=a+mt`. Нужно `mt≡b-a (mod n)`. Пусть `g=gcd(m,n)`. Если `(b-a)%g≠0`, решений нет. Иначе

$$t \equiv \frac{b-a}{g}\,(m/g)^{-1} \pmod{n/g}, \qquad x \equiv a + mt \pmod{\operatorname{lcm}(m,n)}.$$

Обратный элемент существует, поскольку `gcd(m/g,n/g)=1`. Нормализовать остаток: `((x%M)+M)%M`. Частный случай `n/g=1` даёт `t=0`, вычислять обратный по модулю 1 не требуется. При нескольких сравнениях объединять попарно.

Пример КТО. Нужно `x ≡ 3 (mod 4)` и `x ≡ 5 (mod 6)`. Подставляем `x=3+4t`: `4t ≡ 2 (mod 6)`, то есть `2t ≡ 1 (mod 3)`. Получаем `t ≡ 2 (mod 3)`, ответ `x ≡ 11 (mod 12)`. Если вместо второго остатка было 4, решений нет: первое сравнение делает число нечётным, второе — чётным. Именно эту несовместность проверяет НОД.

Почему работает сумма цифр. Для `N=Σ d_i b^i` имеем `b≡1 (mod b−1)`, поэтому `N≡Σd_i (mod b−1)`. В 2023 B можно спросить основания `10^9` и `10^9−1`. Модули `999999999` и `999999998` взаимно просты, их произведение больше `10^15`; КТО однозначно определяет `N∈[1,10^15]`.

Ловушки. Произведение модулей само по себе недостаточно при невзаимной простоте: важен НОК. Для промежуточных произведений нужен `__int128`. Отрицательные коэффициенты Евклида нормальны; C++ `%` для отрицательных чисел не нормализует остаток. Код `extended_gcd_recheck.hpp` возвращает тождество Безу; готового объединения КТО там нет.

T02. НОД как инвариант достижимости

Если состояние меняется на `+w` и `−r`, все достижимые состояния сохраняют остаток по `gcd(w,r)`. Это **не всегда достаточное условие**: промежуточные ограничения тоже надо проверить.

В 2016 B объём `x` лежит в `[0,L]`. Тупик возможен только при `L−w<x<r`. Для этой конкретной модели ответ эквивалентен

$$L < r + w - \gcd(r, w).$$

Пусть `g=gcd(r,w)`. При `L≥r+w−g` интервал `(L−w,r)` не содержит кратных `g`, поэтому тупик невозможен. Обратно, выбираем чтение при `x≥r`, иначе запись. Без ограничения ёмкости оба перехода равны `x→(x+w) mod(r+w)`: чтение даёт `x−r`. Этот цикл из 0 посещает все кратные `g` до `r+w−g`. Если `L` меньше максимума, перед первым выходом за `L` чтение уже невозможно, а запись не помещается — это достижимый тупик.

Ловушки. Границы открытые: при `x=r` читать ещё можно, при `x=L−w` писать ещё можно. Складывать `r+w` при ограничении `10^18` допустимо в `int64`, но в общей заготовке лучше широкий тип.

T03. Линейная алгебра над GF(2) и XOR-базис

Число из `B` битов — вектор над полем `{0,1}`. Сложение — XOR. Каждый базисный вектор хранится по старшему установленному биту.

Вставка / проверка принадлежности. Идём по битам сверху вниз. Если текущий бит `x` равен 1 и в базисе уже есть вектор с этим старшим битом, делаем XOR. Если такого вектора нет — вставляем `x`; при проверке принадлежности это означает неуспех. Остаток 0 означает линейную зависимость / принадлежность. Вставка и запрос — `O(B)`, память `O(B)`; объединение двух базисов — `O(B²)`.

Ранг `r` даёт `2^r` различных XOR. Для максимального XOR можно идти сверху вниз и заменять `x` на `max(x,x xor basis[i])`. Для `k`-го XOR требуется дополнительно привести базис к подходящему каноническому виду: обычная вставка не сортирует множество значений.

Пути в неориентированном графе. Зафиксировать остов и `d[v]` — XOR от корня до `v`. Каждое неостовное ребро `(u,v,w)` даёт цикл `d[u] xor d[v] xor w`. Все XOR **маршрутов с разрешёнными повторами** из `s` в `t` равны

$$d[s] \oplus d[t] \oplus \operatorname{span}(\text{XOR циклов компоненты}).$$

Чтобы использовать удалённый цикл, можно дойти до него и вернуться: соединяющий путь проходит дважды и сокращается по XOR. Для простых путей это рассуждение применять нельзя. Пример — 2022 A.

Гаусс для системы Ax=b над GF(2). Вместе с коэффициентами хранить правую часть. Нулевая строка с правой частью 1 означает противоречие. `k−rank` свободных переменных дают `2^(k−rank)` решений. С `bitset` сложение строк ускоряется в машинное слово.

В 2017 I связность всей решётки позволяет искать окраску `color(z)=c·z mod2`: для каждого разрешённого вектора `v_i` требуется `c·v_i=1`. Проверка бесконечного графа превращается в конечную линейную систему. Важна гарантия связности из условия.

T04. DSU с потенциалами, small-to-large и параллельный бинарный поиск

XOR-потенциалы. Хранить `xr[v]` — XOR от `v` до `parent[v]`. При сжатии пути накопить XOR до корня. Если корни совпали, добавляем цикл `pot[u] xor pot[v] xor w` в базис компоненты. Если присоединяем `root_v` к `root_u`, потенциал нового ребра родителей равен `pot[u] xor pot[v] xor w`; базисы объединяются. Для обычных разностей формулы знаков другие.

Small-to-large. Переносить элементы меньшего множества в большее. При каждом переносе размер содержащего множества как минимум удваивается, поэтому элемент переносится `O(log n)` раз. Это доказательство надо приводить для реального размера структуры, а не просто для номера корня.

Параллельный бинарный поиск. Есть много запросов с монотонным предикатом по порогу `W`. Для каждого храним `[lo,hi]`, группируем очередные `mid`, один раз проходим события в порядке порога и проверяем все запросы. Повторяем `O(log M)` раундов. Предикат и его монотонность доказываются до выбора структуры.

Ловушки. Все рёбра одинакового веса должны быть добавлены до проверки этого порога. Невыполнимые запросы получают отдельный ответ. DSU после прохода обычно нужно заново инициализировать.

T05. Граф состояний, SCC, DAG и потенциалы

Цикл в ориентированном графе. DFS: 0 — не посещена, 1 — в текущем стеке, 2 — завершена. Ребро в цвет 1 обнаруживает цикл. Для восстановления хранить родителей и конкретное ребро; петля — цикл длины 1. Альтернатива: после удаления вершин нулевой входной степени по Кану непустой остаток означает наличие цикла.

SCC. Компонента сильной связности — максимальное множество попарно достижимых вершин. После сжатия компонент получается DAG. Косарайю или Тарьян работают за `O(V+E)`. Внутри SCC размера больше 1 есть ориентированный цикл; одиночной вершине нужна петля. При задаче на подмножество SCC проверять, требуется ли путь внутри подмножества или разрешены внешние промежуточные вершины.

DP по DAG. Если `v` зависит от `u`, обработать `u` раньше `v`. Например, в 2022 E эффективное время `T[v]=max(t[v],max T[u])` по всем зависимостям. Обход с мемоизацией эквивалентен топологическому порядку, но на длинной цепочке рекурсия может переполнить стек.

Равенства потенциалов. Условия `x[v]−x[u]=w` проверяются DFS/BFS: выбрать произвольное значение в компоненте, распространить значения и проверить все рёбра. Разные пути должны давать одинаковый потенциал. Это проще, чем общий алгоритм разностных неравенств. В 2017 G переменные — числа вихрей в клетках тора; обязательно проверить и замыкающие границу рёбра.

T06. Кратчайшие пути и сжатие плотных переходов

Выбор алгоритма: BFS при единичных весах; 0–1 BFS для весов 0/1; Дейкстра для неотрицательных; DAG-DP для ациклического графа, в том числе с отрицательными весами. У Дейкстры с кучей `O((V+E)log V)`. Устаревшие записи кучи пропускать.

Лексикографическая стоимость. Путь имеет цену `(число автобусов, число рейсов)`. Можно хранить пары и сравнивать лексикографически. Сложение покомпонентное, веса неотрицательные, поэтому Дейкстра корректна. Замена пары на `bus*C+flight` допустима лишь после доказательства достаточного `C` и отсутствия переполнения.

Вершина-посредник вместо клики. Если внутри группы из `k` состояний разрешён переход между любой парой за одну и ту же цену, не строить `0(k²)` рёбер. Добавить общий узел: `state→hub` с нужной ценой, `hub→state` с нулевой. В 2024 I терминалы аэропорта соединяются автобусным узлом: вход стоит `(1,0)`, выход `(0,0)`, рейс `(0,1)`.

К источнику соединить все допустимые стартовые состояния бесплатно, аналогично финишу. Посредник должен сохранять именно допустимые маршруты и их цены. Разбиение одной цены по двум дугам требует внимания; две ненулевые половины могут случайно удвоить стоимость.

T07. Паросочетания, теорема Холла и отсечение кандидатов

В двудольном графе паросочетание насыщает левую долю тогда и только тогда, когда для любого её подмножества `S` выполняется `|N(S)|≥|S|` — теорема Холла. Алгоритм Куна ищет увеличивающие пути; общая оценка `O(V_left E)`. Hopcroft–Karp — `O(E√V)`.

Сведение выбора разных объектов. Каждый участник — левая вершина; допустимый объект — правая; выбрать каждому различный объект означает найти насыщающее паросочетание. Минимизация суммы цен правых объектов сводится к min-cost flow: источник ⟶ участник ⟶ объект ⟶ сток, ёмкости 1; стоимость платится один раз при выборе объекта.

Почему иногда хватает `n` кандидатов на человека. Пусть слева `n` вершин, а стоимость зависит только от выбранного правого объекта. Оставим у каждой левой вершины `n` самых дешёвых различных кандидатов. Если в решении использован отброшенный объект, среди сохранённых `n` кандидатов хотя бы один не занят другими `n−1` участниками. Замена не ухудшает цену. Пример — 2024 C, где объекты — непустые подмножества друзей, цена — размер подмножества.

Это не универсальное правило для любых ограничений: нужны независимость допустимости от остальных выборов и указанная функция стоимости. Генерация `n` самых дешёвых кандидатов тоже должна быть эффективной.

T08. Потоки, ограничения по группам и разрезы

Поток подчиняется ёмкостям и сохранению в промежуточных вершинах. Максимальный поток равен минимальной ёмкости `s–t`-разреза. После окончания алгоритма вершины, достижимые из `s` по положительным остаточным ёмкостям, задают одну сторону минимального разреза.

Ёмкость на вершину. Заменить `v` на `v_in→v_out` с ёмкостью `c[v]`; входящие дуги направить в `v_in`, исходящие — из `v_out`. Для запрета разрезать вершину ставят достаточно большое `INF`, которое превышает любой конечный ответ.

Ограничения нескольких уровней. В 2021 L заказ может идти пешком или на машине. Источник ⟶ заказ (`1`), затем к допустимому пешему узлу склада или автомобильному узлу. Автомобильный узел ⟶ общий узел склада (`d`); общий узел ⟶ сток (`c`). Так ограничение `d` вложено в общий лимит `c`; две независимые дуги в сток ошибочно дали бы до `c+d` доставок. Заказы можно добавлять по одному и продолжать поток в сохранённой остаточной сети.

Min-cost flow. Обратное ребро имеет противоположную стоимость и нулевую начальную ёмкость. Для Дейкстры использовать приведённые стоимости `c'(u,v)=c(u,v)+π[u]−π[v]≥0`; после поиска прибавить конечные расстояния к потенциалам. Если есть отрицательные начальные стоимости, нужны корректные стартовые потенциалы. Отрицательные циклы и повторный вызов на уже изменённой сети требуют отдельного контракта.

T09. Деревья: Euler tour, HLD, расстояния и центроиды

Поддерево — отрезок. При DFS каждой вершине назначить время входа `tin[v]`, а `tout[v]` сделать первым временем после поддерева. Тогда поддерево равно `[tin[v],tout[v])`. Если нужны только листья, нумеровать только листья: множество листьев любого поддерева всё равно образует отрезок. Так 2025 B превращает вероятности листьев в массив и операции над отрезками.

LCA. `dist(u,v)=depth[u]+depth[v]−2depth[lca(u,v)]`. Двоичные подъёмы требуют `O(n log n)` памяти и подготовки, запрос — `O(log n)`. Для взвешенного дерева вместо глубины в формуле использовать сумму весов до корня.

HLD. Тяжёлый сын имеет максимальное поддерево. Любой путь содержит `O(log n)` лёгких рёбер: при переходе к лёгкому сыну размер уменьшается как минимум вдвое. Операции на пути разбиваются на `O(log n)` отрезков, отсюда обычно `O(log² n)` на запрос с деревом отрезков.

Диаметр и центр. Два обхода находят диаметр дерева. Центр — одна вершина или два соседних центра диаметра; минимизирует максимальное расстояние до вершин. Это отличается от центроида, после удаления которого все компоненты имеют размер не больше `n/2`.

Центроидная декомпозиция. Каждая вершина имеет `O(log n)` предков в дереве центроидов. Для подсчёта вершин на расстоянии `k` суммировать счётчики расстояний до центроидов и вычитать ветвь, из которой пришёл запрос. Статическая декомпозиция сама по себе не решает задачу с добавлением рёбер между компонентами: 2018 D требует дополнительного механизма перестройки.

T10. Дерево отрезков как моноид; ленивые операции

Нужны ассоциативная операция `merge` и нейтральный элемент. Коммутативность не требуется: порядок левого и правого отрезка важен для строк, матриц и чередующихся сумм.

Пример чередующейся суммы. Храним пару `(s,p)`, где `s=a1−a2+a3−…`, `p=length mod2`. Тогда

$$\mathrm{merge}((s,p),(t,q)) = (s + (-1)^p t,\ p \oplus q).$$

Нейтральный элемент `(0,0)`. Если часть элементов отфильтрована, `p` — чётность числа **оставшихся** элементов, а не длины исходного отрезка. В 2023 G это отдельная пара для каждого возможного числа различных простых делителей. У числа `≤10^6` их максимум 7: произведение первых восьми простых уже больше `10^6`.

Lazy propagation. Отложенная операция должна действовать на агрегат и композиционно объединяться с другими операциями. Для `x→ax+b` и суммы: `sum→a·sum+b·len`. Если сначала применили `(a1,b1)`, затем `(a2,b2)`, итог `(a2a1,a2b1+b2)`. Для assign/add порядок также существенен.

Умножение вероятностей на поддереве в 2025 B требует `range multiply + range sum`. Текущее дерево в nuclear реализует `range add + range max`, поэтому его нельзя использовать без изменения агрегата и тегов.

Ускорение через монотонный рост. Если операция реально умножает положительное число хотя бы на 2, а оно не превосходит `A`, на одну жизнь элемента приходится `O(log A)` изменений. Дерево с минимумом ищет лишь подходящие элементы. Присваивание начинает новую жизнь; `c=1` нужно сразу пропускать. Без этого амортизационная оценка для 2023 G неверна.

T11. Неявный treap и амортизация уменьшением вдвое

Неявный ключ — позиция по размеру левого поддерева. `split(t,k)` делит первые `k` элементов и остаток; `merge(a,b)` конкатенирует последовательности. При независимых случайных приоритетах ожидаемая высота `O(log n)`. Перед спуском — `push`, после изменения детей — `pull`.

Для вырезания `[l,r)` нужны два split: сначала по `l`, затем по `r−l`. Операцию применить к середине и собрать обратно. Реверс меняет местами детей и переключает флаг; для некоммутативного агрегата нужно хранить обе ориентации.

Если по указателю надо узнать позицию элемента, нужны родительские ссылки. Каждый split/merge/поворот обязан поддерживать их; перед вычислением позиции учитывать отложенные операции на пути к корню.

Амортизация уменьшением вдвое. Если целый запас `H` не растёт, а каждая дорогая пересборка уменьшает его хотя бы вдвое, до исчерпания возможны `O(log(H+1))` пересборок. Считать нужно пересборки, а не все мелкие события между ними.

T12. DP с малой памятью и восстановлением ответа

Две соседние прослойки DP сокращают память, но не восстанавливают путь автоматически. Есть два основных варианта.

Упакованные предки. Если у состояния 3 типа переходов, достаточно 2 бит на предка, а не 4/8 байт. Для `S` состояний нужно примерно `S/4` байт плюс рабочие слои. Надо хранить код перехода, из которого однозначно восстанавливаются координаты предыдущего состояния.

Разделение пути посередине. Для задачи на отрезке посчитать прямую и обратную DP на среднем слое. Найти состояние, через которое проходит оптимальный путь, рекурсивно восстановить две части. Альтернатива — при прямом проходе протащить идентификатор состояния пересечения среднего слоя.

Память равна памяти одного прохода плюс рекурсивный стек. Время **не всегда** остаётся исходным: требуется оценить сумму размеров подзадач по уровням. В 2022 I разрез идёт по одному из ресурсов и сохраняет нужную оценку `O(NK1K2)` времени при `O(N·min(K1,K2))` рабочей памяти, выбирая удобную координату для слоя.

Ловушки. Перекрытие участков ремонта, конечные состояния «не более» вместо «ровно», равные оптимумы, нулевой ресурс и подзадачи без перехода через выбранную середину.

T13. Вероятность: индикаторы, первый шаг и самопереходы

Линейность ожидания. `E[ΣX_i]=ΣE[X_i]` без требования независимости. Для индикатора события `E[I]=P(event)`. Если надо посчитать число вершин случайного trie, суммировать вероятности существования каждой возможной вершины. Независимость нужна для произведения вероятностей отсутствия, и события одного исходного выбора могут быть зависимы.

Первый шаг. Если один шаг стоит 1, `E[s]=1+Σ P(s→t)E[t]`. Самопереход с вероятностью `q` переносится влево:

$$E[s] = \frac{1 + \sum_{t \neq s} P(s \to t) E[t]}{1 - q}.$$

Если `q=1`, конечного ожидания нет. При нескольких взаимозависимых состояниях нужна система уравнений, а не произвольный порядок DP. B 2025 E перезаписанный честным случайным битом префикс теряет информацию о старых значениях; это позволяет сжать состояние до длины уже рандомизированного префикса.

Случайный выбор из очереди. B 2016 L удобно считать назад ожидаемый момент удаления заявки `F`. На событии удаления в момент `t`, если до него `k` заявок:

$$F \leftarrow t/k + (k-1)F/k.$$

На событии добавления заявки в момент `t` её ожидание пребывания равно `F−t`. Вставка других заявок не меняет ожидаемый момент для уже присутствующей заявки при обратном проходе.

По модулю. Рациональное `a/b` означает `a·b^{-1} mod p`, только если `b` обратим. Это не вещественное приближение. Для многих известных ненулевых знаменателей `d_i` построить префиксные произведения, обратить одно итоговое произведение и пройти назад: все обратные за `O(n+log p)` вместо `O(n log p)`.

Точный двоичный масштаб. Если вероятности получаются усреднением двух исходов глубины до `D`, можно хранить `2^D·P` целыми. Пример 2022 J: капитал поддерживается пропорциональным вероятности победы; ставка — половина разности капиталов двух следующих состояний.

T14. Факториалы, мультиномиальные коэффициенты и циклы перестановки

Для частот `c_0,…,c_9`, `L=Σc_i`, число различных перестановок равно `L!/∏c_i!`. Без ведущего нуля при `L>0` вычесть

$$\frac{(L-1)!}{(c_0-1)!\prod_{i=1}^{9} c_{(i)!}} \quad (c_0>0).$$

Для пустого префикса число перестановок равно 1; допустимость числа целиком проверяется отдельно. По простому модулю с `L<p` предвычислить `fact,invfact`; когда `L≥p`, простое деление факториалов не работает.

Добавление элемента в цикл. У перестановок `k` элементов новый элемент либо образует отдельный цикл, либо вставляется после одного из старых `k` элементов. Отсюда беззнаковые числа Стирлинга первого рода:

$$c(k+1,j)=c(k,j-1)+k\,c(k,j), \quad \sum_{j} c(k,j)x^j = x(x+1)\cdots(x+k-1).$$

T15. Свёртки, NTT и произведение многочленов

Если `A(x)=Σa_i x^i`, `B(x)=Σb_j x^j`, то `[x^k]AB=Σ_{i+j=k}a_i b_j`. Таким образом пары с заданной суммой считаются свёрткой. Для разностей развернуть один массив; отрицательные координаты сдвинуть и внимательно пересчитать индекс ответа.

NTT. Для длины `N=2^k` в поле `mod p` нужен примитивный корень порядка `N`, поэтому `N|(p−1)`. Для `998244353=119·2^23+1` подходят степени двойки до `2^23`, стандартный примитивный корень — 3. `N` выбирают не меньше `len(A)+len(B)−1`, иначе получится циклическая свёртка. Обратное преобразование использует обратный корень и умножение на `N^{-1}`. Время `O(N log N)`.

Точное число, а не остаток. Если коэффициенты могут быть больше модуля, одной NTT недостаточно. Нужны несколько простых модулей и КТО с произведением выше границы коэффициента либо обоснованное восстановление через FFT.

T16. Неравенства, транспорт на прямой и допустимые суммы

Коши–Буняковский для времени и штрафа. На фиксированном маршруте с длинами `l_i`, ценами `c_i` и временами `t_i>0` минимизируется `Σc_i l_i/t_i` при `Σt_i=T`. Тогда

$$\left(\sum_i \frac{c_i l_i}{t_i}\right)\left(\sum_i t_i\right) \ge \left(\sum_i \sqrt{c_i l_i}\right)^2.$$

Равенство при `t_i=T·sqrt(c_i l_i)/Σsqrt(c_j l_j)`. В 2021 M осталось найти кратчайший путь с весом `sqrt(c_i l_i)`, затем возвести расстояние в квадрат и разделить на `T`. Произведение `c_i l_i` вычислять после преобразования в достаточно широкий тип.

Землекопная метрика на цепочке. Чтобы перевести распределение `a` в `b` соседними переносами единичных предметов, через ребро `i|i+1` неизбежно переносится `|A_i−B_i|`, где `A_i=Σ_{j≤i}a_j`. Поэтому

$$W(a,b)=\sum_{i=1}^{n-1} |\,A_i-B_i\,|\ .$$

Для нескольких распределений оптимальный `B_i` — медиана их префиксных сумм. Координатные медианы неубывающих последовательностей сами неубывают, значит `b_i=B_i−B_{i−1}≥0`. Это завершает доказательство 2020 G; независимая минимизация без проверки допустимости была бы неполной.

Липшицевы ограничения. Если `|t_i−t_{i+1}|≤1`, то между фиксированными `(a,A),(b,B)` сначала проверить `|A−B|≤b−a`. Для внутренней позиции `i`:

$$\max(0,A-(i-a),B-(b-i)) \le t_i \le \min(A+i-a,B+b-i).$$

Суммы этих «долин» и «гор» считать арифметическими прогрессиями. В 2017 H все целые суммы между минимумом и максимумом достижимы: если допустимая последовательность ещё не максимальна, можно поднять подходящую недофиксированную впадину на 1, сохранив ограничения. Краевые интервалы имеют только одну фиксированную границу.

T17. Строки: границы, подстроки и палиндромы

Граница строки — её префикс, совпадающий с суффиксом. В префикс-функции нужна собственная граница, короче всей строки. **Префикс-функция** `π[i]` — длина максимальной собственной границы префикса `s[0..i]`. Переходы по `π` дают поиск образца за `O(n+m)`. **Z-функция** `z[i]` — LCP всей строки и суффикса с позиции `i`; для поиска в `pattern + sep + text` разделитель не должен встречаться во входе.

Дерево границ. У префикса длины `j>0` родитель — `pi[j-1]`. Предки состояния KMP `L`, включая сам `L`, дают длины префиксов образца, совпадающих с суффиксом прочитанного текста. Условие «`j` — предок `L`» превращается в попадание `tin[L]` в Euler-отрезок поддерева `j`. Два таких условия дают прямоугольник на двух Euler-порядках; офлайн их считают sweep и Fenwick.

Суффиксные структуры. Число различных подстрок через SAM равно `Σ(len[v]−len[link[v]])` по неначальным состояниям. Через suffix array:

n(n+1)/2−ΣLCP. LCP двух суффиксов получается RMQ между их рангами. Выбор между SAM и SA зависит от запросов, а не только от наличия кода.

Манакер. Радиусы нечётных и чётных палиндромов находятся за O(n). Для бинарной строки чётная подстрока является антипалиндромом, если симметричные биты различны. Преобразование t[i]=s[i] xor (i mod2) превращает чётные антипалиндромы в обычные чётные палиндромы. Это позволяет использовать стандартного Манакера в 2025 E.

T18. Геометрия: ориентация, отражения, оболочки и сечения

В 2D cross(a,b)=a_x b_y−a_y b_x; знак cross(b−a,c−a) задаёт ориентацию тройки. dot(a,b) определяет проекцию. Для целочисленных координат выбирать тип по **промежуточному** произведению, а не только по входу; для геометрических сравнений знак часто можно вычислять точно.

Отражение о прямую. Пусть прямая проходит через a, имеет ненулевой направляющий вектор d. Проекция h=a+d·dot(p−a,d)/dot(d,d), отражение p'=2h−p. Отсечение выпуклого многоугольника полуплоскостью: обойти рёбра, оставить внутренние вершины и добавить пересечения при смене стороны.

Опорная функция оболочки. h_K(d)=max_{p∈K}dot(p,d). Линейный функционал достигает экстремума на выпуклой оболочке, поэтому внутренние точки не нужны. Для статической выпуклой оболочки экстремум по направлению можно искать логарифмически, тщательно обработав равные значения и циклический порядок. Для 2022 D оболочка динамическая: код одного построения Грэхема недостаточен.

Сечения площадей. Площадь — интеграл длины вертикального сечения. Если сумма площадей треугольников меньше площади прямоугольника, найдётся вертикаль, на которой сумма длин сечений меньше высоты. Длина сечения треугольника кусочно линейна по x; суммарную функцию поддерживают событиями в абсциссах вершин. Найдя хорошую вертикаль, объединить отрезки сечений и взять точку в зазоре. Это детерминированный путь к 2017 J; случайная точка без оценки площади зазора может искаться слишком долго.

T20. Конечные игры и малые грамматики

В конечной игре без ничьих состояние выигрышно, если есть ход в проигрышное; проигрышно, если все ходы ведут в выигрышные. Терминальная позиция без ходов проигрышна при обычном правиле игры. Считать в порядке, обратном направлению ходов; если возможны циклы, простая рекурсия win/lose уже недостаточна.

Римские числа — конечный язык. Канонических чисел от 1 до 3999 всего 3999; их можно сгенерировать таблицами для тысяч, сотен, десятков и единиц. В 2023 L ход добавляет букву, длина строго растёт. Генерация всех строк и DP по убыванию длины дают полную стратегию, включая выбор, кто ходит первым.

В 2021 H оставить из словаря только палиндромы и решать разбиение строки: dp[i]=1+min dp[i−|w|] по словам w, совпадающим с суффиксом. Запомнить выбранное слово для восстановления. Это конечный словарь и DP, а не общая сложная задача о палиндромном разбиении.

Ловушки. Различать каноническую запись и произвольную строку, которую можно арифметически интерпретировать; запрещены неканонические формы. Интерактивный протокол завершается сразу после ответа об окончании игры.

T21. Интерактив: информация, устойчивые сравнения и бюджет

До реализации записать: что фиксировано, может ли жюри адаптироваться, точные ответы при равенстве, ограничения на каждый запрос и суммарный бюджет. После запроса — перевод строки и flush; при EOF или специальном ошибочном ответе немедленно завершить работу.

Информационная граница. Если осталось M равноправных кандидатов и каждый ответ имеет не больше q вариантов, глубина дерева решений не меньше ceil(log_q M). Это необходимое, но не достаточное условие: разрешённые вопросы могут плохо делить кандидатов. Для 2022 L выгодно заранее перебрать малые перестановки и построить допустимое дерево сравнений.

T24. Арифметические функции, факторизация и массовая обработка

Если n=∏p_i^α_i, то

$$\tau(n)=\prod_i(\alpha_i+1),\qquad \sigma_1(n)=\prod_i(1+p_i+\cdots+p_i^{\alpha_i}).$$

Геометрическую сумму по модулю не следует всегда писать делением на p_i−1: знаменатель может быть необратим. Для факториала показатель простого — формула Лежандра v_p(n!)=Σ_{j≥1}floor(n/p^j). Для чётного двойного факториала (2t)!!=2^t t!; нечётный двойной факториал нечётен. Это сразу ограничивает возможный k! в 2025 D.

Делители парами. Если d|n и d²<n, к сумме добавляются d+n/d; при d²=n один делитель. В 2025 K по отдельности факторизовать θ(n²) сумм дорого. Для фиксированного d условие d|(a_i+a_j) означает противоположные остатки r и −r mod d. Группировка остатков и два указателя обрабатывают много пар совместно. Особые группы r=0 и, при чётном d, r=d/2 требуют считать неупорядоченные пары один раз.

Простота против факторизации. Miller–Rabin проверяет простоту, но не выдаёт разложение на множители.

8 Идеи решения задач

8.1 Повторяющиеся приёмы московских контестов

Приёмы, которые полезно узнавать в условии

Здесь собраны способы думать о задаче. Их стоит читать вместе с теорией: название структуры данных помогает только после того, как найдена модель. Полные условия и карта всех задач находятся в `docs/moscow`.

Использовать необычную гарантию как часть алгоритма

В 2020 L матрица огромна, зато интервалов не больше `n+300`: после перехода к графу это означает всего 300 лишних рёбер сверх дерева. В 2021 C цветов много, зато у каждого не больше пяти пар носков: все пары цветов описываются таблицей размера 5 на 5. Полезный вопрос: какое ограничение пока вообще не использовалось в рассуждении?

Исправлять только то место, где нарушился инвариант

В Christmas Children Circle (2021 D) после передачи чисел в мешках не должно быть повторов. Сначала решаем задачу для линии: передаём входящее число дальше, если оно уже есть в мешке. После такого прохода неверным может быть только переход от последнего ребёнка к первому. Последнее число распространяем по начальному отрезку мешков, где оно есть. На первой дыре старый ответ снова подходит. Получилось два прохода вместо перебора всех стартовых вариантов.

Не искать больше, чем требуется доказательству

В аирдропах 2023 A хочется искать глобально ближайшую пару точек, что дорого при повторении. Достаточно взять одну любую точку и ближайшую к ней. Расстояние считаем по Манхэттену. В целую середину пути никто не придёт раньше выбранной точки; за дополнительную секунду решения задачи туда успевает и её сосед. Полезная замена: искать объект, для которого выполнено нужное неравенство, вместо глобального оптимума.

Отдельно посчитать цену неизбежного и цену взаимодействий

В Socks Drying (2021 C) `a[i]` — число пар носков цвета `i`. Извлечения и сравнения внутри найденных пар дают базовые `3 sum(a[i])` секунд. Дополнительные сравнения относятся к парам разных цветов. Их относительный порядок случаен независимо от того, сколько носков других цветов находится между ними. Поэтому огромный процесс заменяется малой DP для двух цветов и суммированием парных вкладов.

Обосновывать жадный выбор конкретным обменом

В Judge Problem (2018 J) обрабатываем задачи от трудных к лёгким и предпочитаем автора. Если в оптимальном решении автор занят будущей задачей, меняем их местами: прежний обычный исполнитель текущей задачи имеет достаточный порог и для будущей. Так доказывается каждый шаг, а не только интуиция «автору приятнее делать своё». Для обмена надо назвать оба объекта и проверить все ограничения после перестановки.

Превращать правила различимости в маленькие состояния

В Toll Road (2019 F) важны не все истории проехавших машин, а непомеченные въезды и выезды между дорожными воротами. Там допустимы только четыре состояния: нет таких точек, один въезд, один выезд, выезд перед въездом. Пятый вариант — въезд перед выездом — оставил бы невидимую поездку. Когда условие требует различать все пары объектов, сначала спросите, какие две локальные конфигурации дают одинаковое наблюдение.

Проверять, не определяется ли ответ двумя краями

Непослушная скобочная последовательность содержит поровну `(` и `)`, но имеет отрицательный баланс префикса. В 2019 H для разбиения правильной последовательности на такие части нужны только начальная серия `(` длины `L` и конечная серия `)` длины `R`. Критерий: `L+R <= n/2-2`. Вся середина нужна для доказательства существования двух частей, но не для ответа на запрос. Это повод после вывода критерия убрать из структуры данных всю лишнюю информацию.

Считать ответ вкладом элемента, а не перебором объектов

В You Are Given a Tree (2019 J) вместо перебора всех интервалов меток суммируем, в скольких минимальных поддеревьях для этих интервалов присутствует каждая вершина. Она отсутствует, когда все метки интервала лежат в одной компоненте после её удаления. Число интервалов внутри набора меток — сумма `len*(len+1)/2` по непрерывным сериям. Замена направления суммирования открывает small-to-large.

Искать дополнение с единственным свидетелем

В 2020 N неудобно прямо требовать, чтобы оба цвета охватывали центр. Плохой цвет помещается в открытую полуокружность, и у непустого плохого множества есть единственный якорь после большого зазора. Это позволяет считать плохие события по якорям и затем применить включения–исключения. Если считать «все подходящие полуокружности», одно множество учтётся много раз. Сначала докажите единственность свидетеля.

Разделять события с одинаковым временем

В 2019 E программы разрешено использовать по строго возрастающим версиям. Поэтому изменения одной версии сначала вычисляем из старого слоя и только потом применяем вместе. Если обновлять сразу, следующий объект той же версии увидит уже обновлённое состояние и появится запрещённая цепочка. Этот приём повторяется в DP по времени, расстоянию и стоимости.

Убирать общее движение сменой системы координат

Если все объекты одинаково двигаются, хранить можно их положение относительно движущейся рамки. Тогда массовый сдвиг превращается в изменение одной общей величины, а личные события остаются локальными. Например, `x_real = x_stored + shift`: общий сдвиг на `d` меняет только `shift += d`. Для сравнения с реальной границей `X` используем сохранённую границу `X - shift`.

Идти назад, когда будущее проще прошлого

В Lazy Coordinator (2016 L) прямое ожидание срока каждой заявки зависит от всех будущих добавлений и случайных удалений. Обратный проход хранит одну величину — ожидаемый момент удаления присутствующей заявки. На удалении это среднее «удалена сейчас / пережила событие», на добавлении можно сразу выдать ответ. Обратный порядок особенно полезен, когда прямая операция теряет информацию, а обратная позволяет поддерживать одно общее состояние.

Раскрывать максимум до выбора структуры

В Dynamic Traffic (2019 K) переход имеет вид `max_j(dp[j] + max(a[j],x))`. Раскрываем два максимума: `max(max_j(dp[j]+a[j]), x+max_j(dp[j]))`. Получается два числа вместо дерева отрезков и поиска по порогу. Прежде чем поддерживать кусочную функцию, попробуйте переставить `max`, сумму и константу, аккуратно сохраняя диапазон индекса.

Свести бесконечный выбор к конечному автомату

Для римских чисел 1–3999 достаточно сгенерировать все канонические записи. Для НОД анаграмм в 2021 E достаточно маски использованных цифр и остатка по маленькому модулю. BFS с цифрами по возрастанию находит сначала короткое, а среди равной длины — лексикографически минимальное число. Важно доказать, что состояние хранит всё, от чего зависит допустимость продолжения.

Сначала получить границу, затем строить ответ

В конструктивных задачах удобно разделить два доказательства: лучше этого числа быть не может; наше построение достигает этого числа. Например, для Non-Cactus (2021 F) связный граф с единственным циклом ещё является кактусом, поэтому нужно хотя бы `n+1` рёбер. Подходящая основа — два треугольника с общим ребром, остальные вершины подвешиваются листьями. Для маленьких `n` основа не помещается.

Использовать исчерпывающий перебор для малой локальной части

Если остаток конструкции содержит пять объектов или язык имеет 3999 слов, полный перебор может быть надёжной частью основного решения. В Limited Shuffle (2022 L) малые перестановки позволяют заранее построить дерево сравнений. Это отличается от попытки угадать общую закономерность по малым тестам: надо доказать, что вся неограниченная часть уже сведена к этому конечному остатку.